

Ex3_0_RandonVariablesExperiments-solution

Prof M.Tognolini HEIG-VD

February 15, 2026

1 Random Variables experiments

Prof. Maurizio Tognolini - 2026 HEIG-VD - MSE - Statistical Digital Signal Processing rev 1.1 - 2026-02-15

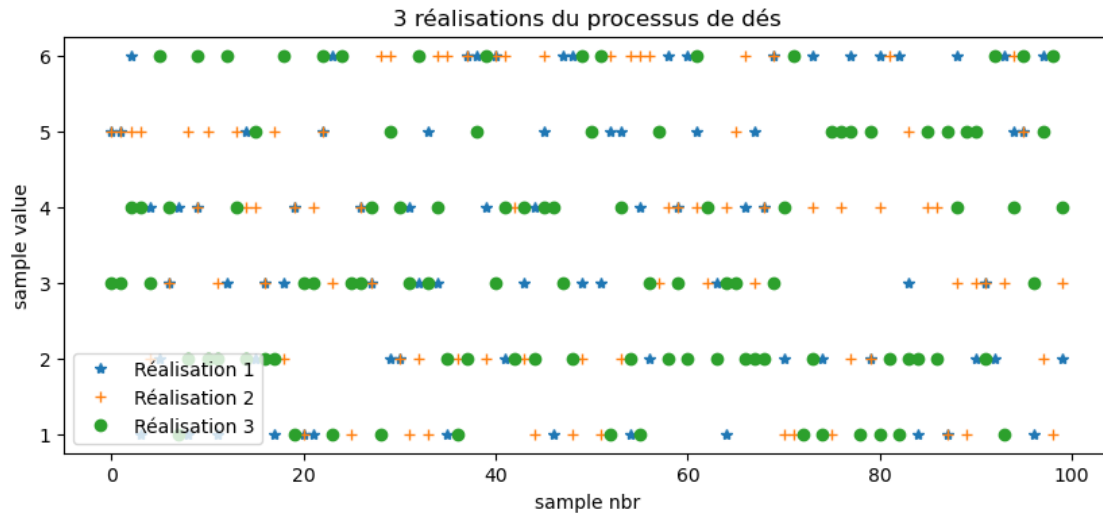
We will simulate roll die with several dies $NR = 50$ (as number of realizations) and we have $N = 100$ samples for each die. Use the function `rand()` to generate a matrix X with sample values (from 1 to 6) , with the column represents sample number and line th realization number. This is a uniform distribution :

```
[ ]: import numpy as np
import matplotlib.pyplot as plt
from scipy import signal

# Paramètres
xmin, xmax = 0.5, 6.5
N = 100    # Nombre d'échantillons
NR = 50    # Nombre de réalisations

# Génération du processus de lancer de dés (Uniforme discret)
# np.random.rand(NR, N) génère directement la matrice au bon format
x = np.round(xmin + (xmax - xmin) * np.random.rand(NR, N))

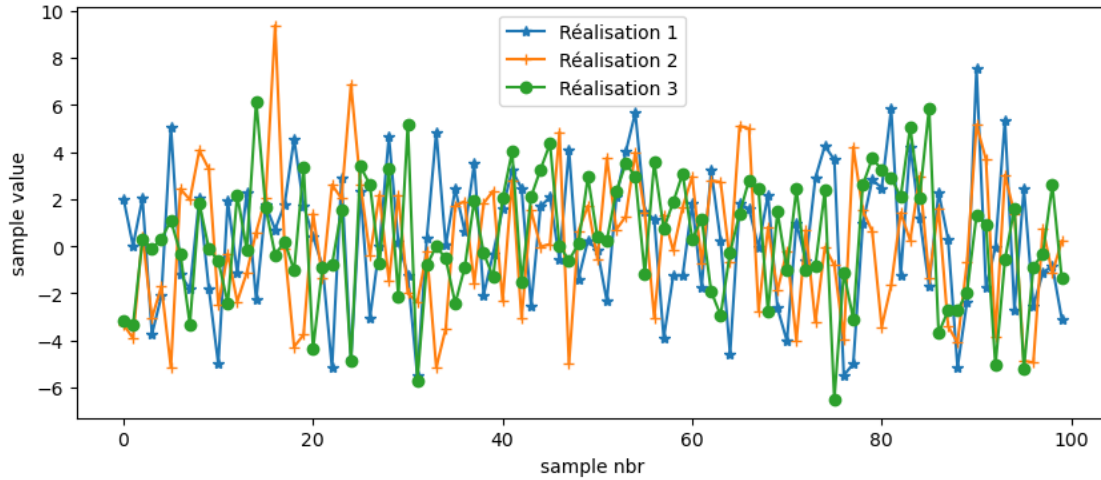
# Affichage de 3 réalisations
plt.figure(figsize=(10, 4))
plt.plot(x[0, :], '*', label='Réalisation 1')
plt.plot(x[1, :], '+', label='Réalisation 2')
plt.plot(x[2, :], 'o', label='Réalisation 3')
plt.legend()
plt.xlabel('sample nbr')
plt.ylabel('sample value')
plt.title('3 réalisations du processus de dés')
plt.show()
```



Alternatively generate another process Y centered to with a standard value of $A=3$ with the function `randn()`, thos is a normal (Gaussian) process. We want also a number of realization NR and a number of samples N :

```
[ ]: A = 3
# Génération d'un processus normal (Gaussien)
y = A * np.random.randn(NR, N)

# Affichage de 3 réalisations
plt.figure(figsize=(10, 4))
plt.plot(y[0, :], '-*', label='Réalisation 1')
plt.plot(y[1, :], '-+', label='Réalisation 2')
plt.plot(y[2, :], '-o', label='Réalisation 3')
plt.legend()
plt.xlabel('sample nbr')
plt.ylabel('sample value')
plt.show()
```



1.1 First order moments in Realization space

For each process estimate the first order moments (mean, and variance) for each realization For the die for each die realization we compute the mean and the std for all the samples values:

```
[83]: # Statistiques pour le processus des dés
muxDie = np.mean(x, axis=1) # Moyenne par ligne

stdxDie = np.std(x, axis=1, ddof=1) # std avec N-1
# --- Affichage des 8 premières réalisations ---
print(f"{'Réalisation':<15} | {'Moyenne (muxDie)':<18} | {'Écart-type (stdxDie)':<20}<20}")
print("-" * 60)
for i in range(8):
    print(f"{i+1:<15} | {muxDie[i]:<18.4f} | {stdxDie[i]:<20.4f}")

# Statistiques pour le processus Gaussien
muyR = np.mean(y, axis=1)
stdyR = np.std(y, axis=1, ddof=1)

# Moyenne des estimations (Espérance de la moyenne)
E_m_die = np.mean(muxDie)
Std_m_die = np.std(muxDie, ddof=1) # Écart-type de la moyenne estimée

print(f"E[muxDie] = {E_m_die:.4f}, Std[muxDie] = {Std_m_die:.4f}")

# Moyenne des estimations (Espérance de la deviation standard)
E_std_die = np.mean(stdxDie)
# variation standard de l'estimation de la deviation standard
```

```
Std_std_die = np.std(stdxDie, ddof=1)

print(f"E[stdxDie] = {E_std_die:.4f}, Std[stdxDie] = {Std_std_die:.4f}")
```

Réalisation	Moyenne (muxDie)	Écart-type (stdxDie)
1	3.6000	1.7809
2	3.5500	1.7717
3	3.4000	1.6453
4	3.3400	1.6650
5	3.1500	1.6659
6	3.4500	1.6104
7	3.5000	1.6907
8	3.4700	1.8448

E[muxDie] = 3.4698, Std[muxDie] = 0.1671
E[stdxDie] = 1.6937, Std[stdxDie] = 0.0633

For the Gaussian process we compute the mean and the std for each realization:

```
[84]: # --- Calcul des moments du premier ordre pour y ---
# muyR : Moyenne de chaque réalisation (moyenne par ligne)
# stdyR : Écart-type de chaque réalisation (std par ligne)
# ddof=1 pour correspondre au calcul non biaisé de MATLAB
muyR = np.mean(y, axis=1)
stdyR = np.std(y, axis=1, ddof=1)

# --- Affichage des 8 premières réalisations ---
print(f"{'Réalisation':<15} | {'Moyenne (muyR)':<18} | {'Écart-type (stdyR)':<20}")
print("-" * 60)
for i in range(8):
    print(f"{i+1:<15} | {muyR[i]:<18.4f} | {stdyR[i]:<20.4f}")

# Moyenne des estimations (Espérance de la moyenne)
E_m_y = np.mean(muyR)
Std_m_y = np.std(muyR, ddof=1) # Écart-type de la moyenne estimée

print(f"E[muyR] = {E_m_y:.4f}, Std[muyR] = {Std_m_y:.4f}")
```

Réalisation	Moyenne (muyR)	Écart-type (stdyR)
1	0.3382	2.8883
2	0.1296	2.9042
3	0.3053	2.6196
4	-0.1043	2.9067
5	0.3121	3.2625
6	0.3586	2.8079
7	-0.2625	2.9948
8	0.3092	2.7813

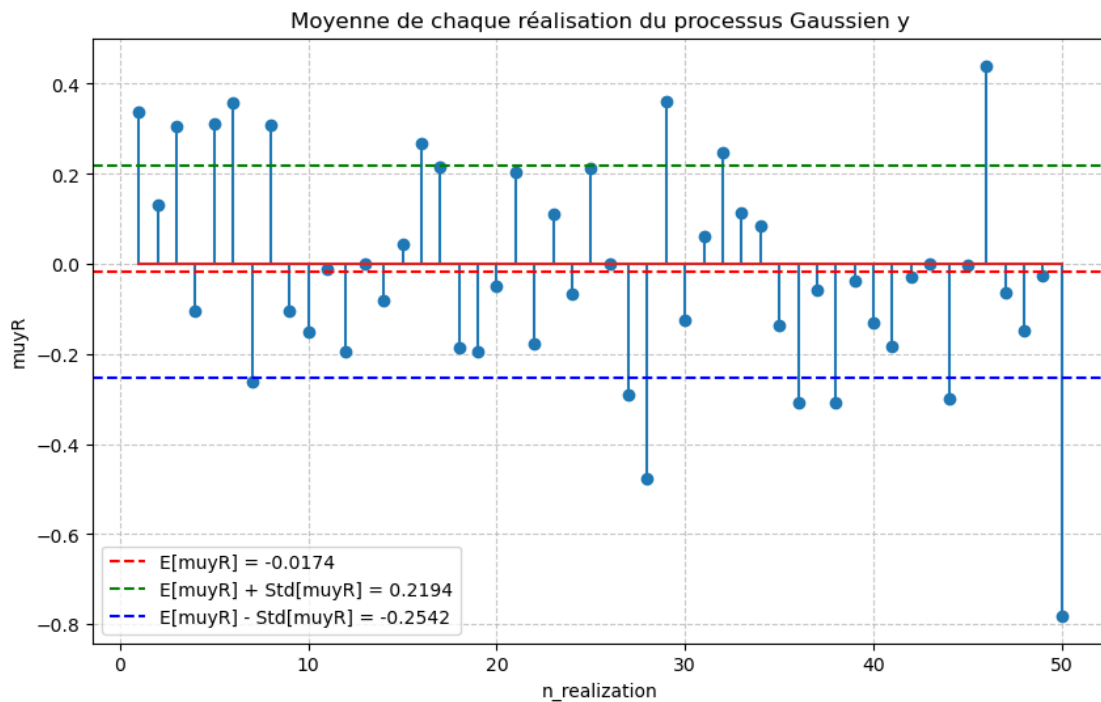
$E[muyR] = -0.0174$, $Std[muyR] = 0.2368$

Calcul de la moyenne pour chaque réalisation (déjà effectué précédemment) $muyR = np.mean(y, axis=1)$

```
[85]: # Création du vecteur pour l'axe des abscisses (de 1 à NR)
n_realization = np.arange(1, NR + 1)

plt.figure(figsize=(10, 6))
# Création du graphique stem (équivalent à stem(n_realization, muyR) en Matlab)
plt.stem(n_realization, muyR)
# tracer une ligne horizontale pour la valeur de  $E[muyR] = -0.0032$ ,
# et deux lignes horizontales a en dessus et dessous de la moyenne
#  $\rightarrow Std[muyR] = 0.2718$ 
plt.axhline(E_m_y, color='red', linestyle='--', label=f'E[muyR] = {E_m_y:.4f}')
plt.axhline(E_m_y + Std_m_y, color='green', linestyle='--', label=f'E[muyR] +
     $\rightarrow Std[muyR] = {E_m_y + Std_m_y:.4f}$ ')
plt.axhline(E_m_y - Std_m_y, color='blue', linestyle='--', label=f'E[muyR] -
     $\rightarrow Std[muyR] = {E_m_y - Std_m_y:.4f}$ ')
plt.legend()
# Ajout des labels et du titre conformément au document
plt.xlabel('n_realization')
plt.ylabel('muyR')
plt.title('Moyenne de chaque réalisation du processus Gaussien y')
plt.grid(True, linestyle='--', alpha=0.7)

plt.show()
```



Calcul de l'écart-type pour chaque réalisation du processus y

```
[86]: # ddof=1 pour correspondre au calcul non biaisé (N-1) de MATLAB
stdyR = np.std(y, axis=1, ddof=1)

# Affichage des 8 premières valeurs avec 4 décimales
print(f"{'Réalisation':<15} | {'Écart-type (stdyR)':<20}")
print("-" * 40)
for i in range(8):
    print(f"{i+1:<15} | {stdyR[i]:<20.4f}")

# Moyenne des estimations (Espérance de la deviation standard)
E_std_y = np.mean(stdyR)
# variation standard de l'estimation de la deviation standard
Std_std_y = np.std(stdyR, ddof=1)

print(f"E[stdxy] = {E_std_y:.4f}, Std[stdxy] = {Std_std_y:.4f}")
```

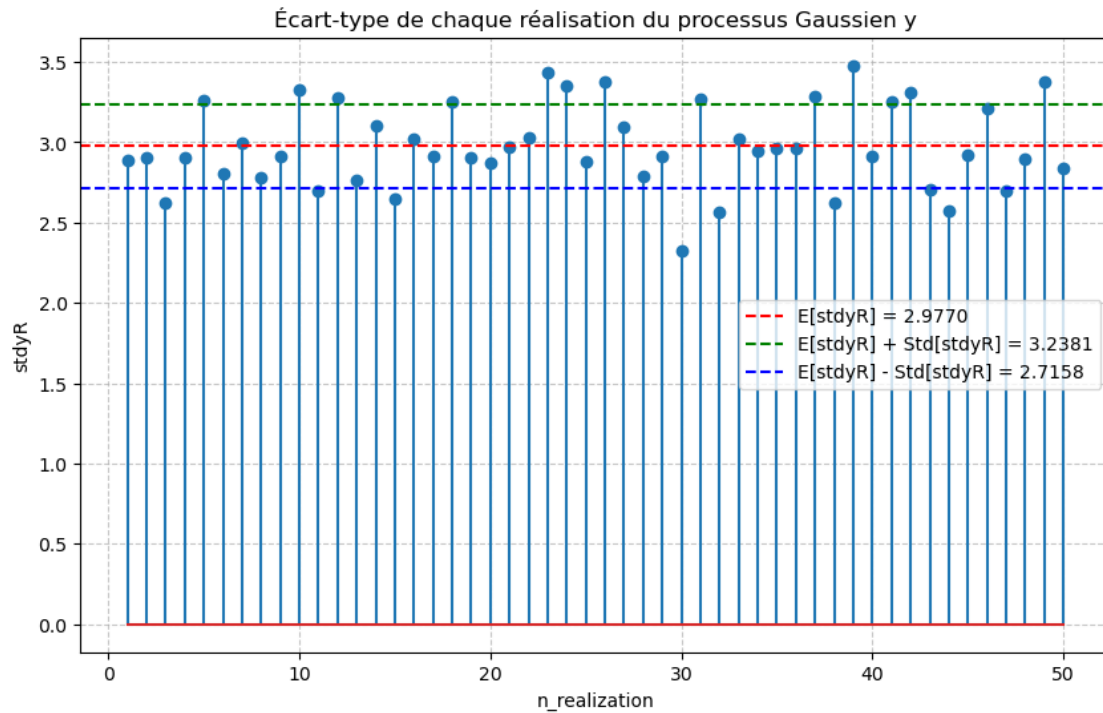
Réalisation	Écart-type (stdyR)
1	2.8883
2	2.9042
3	2.6196
4	2.9067
5	3.2625
6	2.8079
7	2.9948
8	2.7813

E[stdxy] = 2.9770, Std[stdxy] = 0.2611

```
[ ]: plt.figure(figsize=(10, 6))
# Création du graphique stem pour l'écart-type
plt.stem(n_realization, stdyR)
# tracer une ligne horizontale pour la valeur de E[stdxy] = 2.9746,
# et deux lignes horizontales a en dessus et dessous de la moyenne
↳ Std[stdxy] = 0.1846
plt.axhline(E_std_y, color='red', linestyle='--', label=f'E[stdyR] = {E_std_y:.4f}')
plt.axhline(E_std_y + Std_std_y, color='green', linestyle='--', label=f'E[stdyR] + Std[stdyR] = {E_std_y + Std_std_y:.4f}')
plt.axhline(E_std_y - Std_std_y, color='blue', linestyle='--', label=f'E[stdyR] - Std[stdyR] = {E_std_y - Std_std_y:.4f}')
plt.legend()
# Ajout des labels et du titre conformément au document
plt.xlabel('n_realization')
```

```
plt.ylabel('stdyR')
plt.title('Écart-type de chaque réalisation du processus Gaussien y')
plt.grid(True, linestyle='--', alpha=0.7)

plt.show()
```



1.2 Estimation of the density of probability by the hysrogram:

For x random process compute the hystogram on all the sample of all the realizations , this histogram is the discrete estimation of the probability density px we call pxEst , and is the counts of the hystogram divided by the total number of samples $N \cdot NR$.

```
[88]: # 1. Calcul de l'histogramme sur l'ensemble des échantillons (N * NR)
# x.flatten() ou x.ravel() permet d'accéder à toutes les valeurs
# On définit les bacs (bins) pour centrer les valeurs sur [1, 2, 3, 4, 5, 6]
bins = np.arange(1, 8) # Créé [1, 2, 3, 4, 5, 6, 7] pour délimiter les faces
counts, bin_edges = np.histogram(x.flatten(), bins=bins)

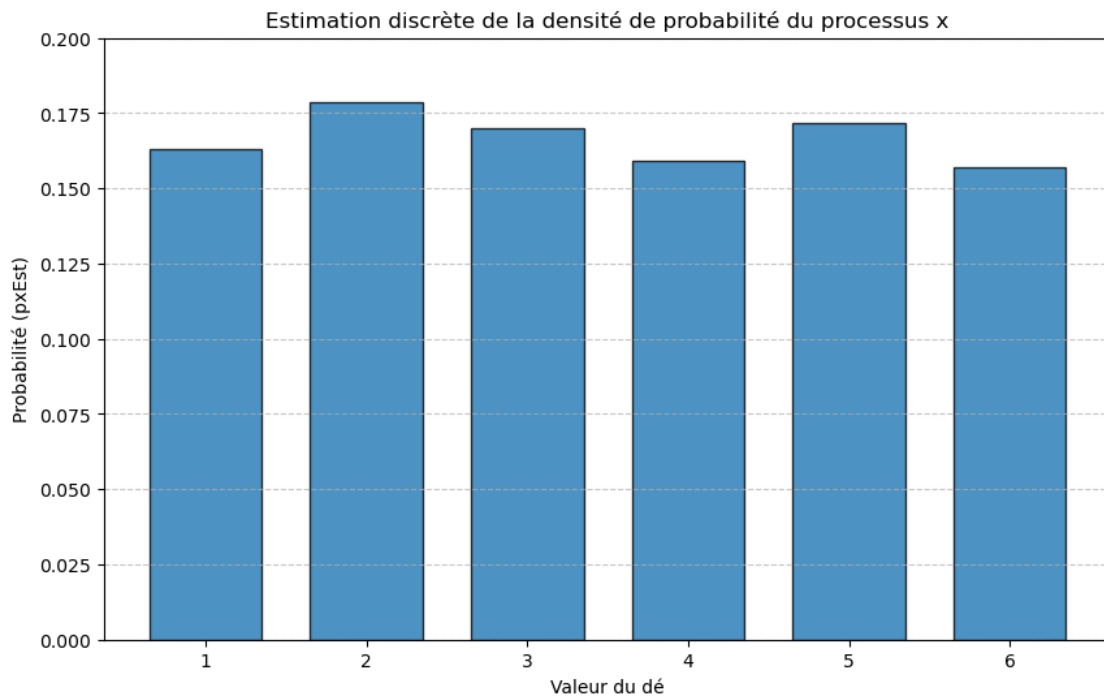
# 2. Estimation de la densité de probabilité discrète (pxEst)
# On divise le nombre d'occurrences par le nombre total d'échantillons
pxEst = counts / (N * NR)

# 3. Affichage graphique (équivalent à bar en Matlab)
plt.figure(figsize=(10, 6))
```

```
plt.bar(np.arange(1, 7), pxEst, width=0.7, edgecolor='black', alpha=0.8)

# Configuration des axes et titres
plt.xlabel('Valeur du dé')
plt.ylabel('Probabilité (pxEst)')
plt.title('Estimation discrète de la densité de probabilité du processus x')
plt.xticks(np.arange(1, 7))
plt.ylim(0, 0.20) # Ajusté pour correspondre au graphique
plt.grid(axis='y', linestyle='--', alpha=0.7)

plt.show()
```



We observe is a discrete uniform distribution! The sum of all pxEst values over all the values has to be equal 1 , since the prob density of y has an integral equal 1: With the function sum() prove it:

```
[89]: # Vérification : La somme doit être égale à 1
integral_pxEst = np.sum(pxEst)
print(f"Somme de pxEst (Intégrale) : {integral_pxEst:.4f}")
```

Somme de pxEst (Intégrale) : 1.0000

For y random process compute the hystogram on all the sample of all the realizations , this histogram is the discrete estimation of the probability density py we call pyEst , and is the counts of the histogram divided by the total number of samples N*NR.


```
[90]: # 1. Calcul de l'histogramme sur tous les échantillons du processus y
# y.flatten() transforme la matrice (NR, N) en un vecteur unique
# On demande 100 classes (bins) comme dans le code MATLAB
county, bin_edges = np.histogram(y.flatten(), bins=100)

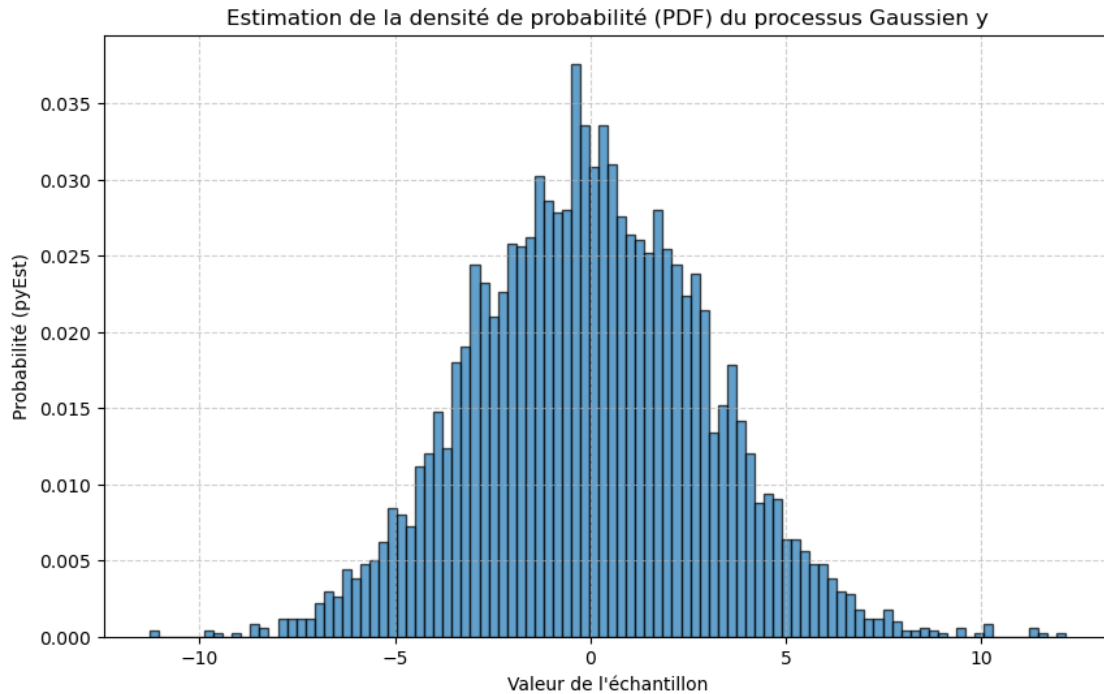
# 2. Calcul des centres des classes (centy en MATLAB)
# np.histogram donne les bords des classes, on calcule le milieu de chaque
    ↪ intervalle
centy = (bin_edges[:-1] + bin_edges[1:]) / 2

# 3. Estimation de la densité de probabilité (pyEst)
# Division du nombre d'occurrences par le nombre total d'échantillons (N * NR)
pyEst = county / (N * NR)

# 4. Affichage graphique avec bar()
plt.figure(figsize=(10, 6))
plt.bar(centy, pyEst, width=(bin_edges[1] - bin_edges[0]), color='tab:blue',
    ↪ edgecolor='black', alpha=0.7)

# Configuration des axes et titre
plt.xlabel('Valeur de l\'échantillon')
plt.ylabel('Probabilité (pyEst)')
plt.title('Estimation de la densité de probabilité (PDF) du processus Gaussien,
    ↪ y')
plt.grid(True, linestyle='--', alpha=0.6)

plt.show()
```



We observe is a discrete gaussian distribution! The sum of all pyEst values over all the values has to be equal 1 , since the prob density of y has an integral equal 1: With the function sum() prove it:

```
[91]: # Vérification : La somme des probabilités doit être égale à 1
print(f"Somme de pyEst (Intégrale discrète) : {np.sum(pyEst):.4f}")
```

Somme de pyEst (Intégrale discrète) : 1.0000

1.3 First order moments in the Sample Space

Now we could compute the mean and std estimate through the realitation for each sample , for exemple the mean est. of the sample nb 1 through all the NR realization is the mean of all the value of the sample nb 1 for all the realization, and so on for all the samples.

```
[92]: # Calcul de la moyenne pour chaque échantillon n (à travers les NR réalisations)
# mxS
mxS = np.mean(x, axis=0)
#print(f"Moyenne de chaque échantillon (mxS) pour la réalisation no 0 à 4 :↳
↳{mxS[0:5]}")
# Calcul de l'écart-type pour chaque échantillon n
# stdxS correspond à std(x) en MATLAB (ddof=1 pour N-1)
stdxS = np.std(x, axis=0, ddof=1)
#print(f"Écart-type de chaque échantillon (stdxS) pour la réalisation no 0 à 4:↳
↳{stdxS[0:5]}")
```

```
# Affichage des 8 premiers échantillons
print(f"{'Échantillon (n)':<15} | {'Moyenne (mxS)':<18} | {'Écart-type (stdxS)':<20}")
print("-" * 60)
for i in range(8):
    print(f"{i:<15} | {mxS[i]:<18.4f} | {stdxS[i]:<20.4f}")
```

Échantillon (n)	Moyenne (mxS)	Écart-type (stdxS)
0	3.4200	1.6424
1	3.2400	1.6728
2	3.3400	1.6113
3	3.2600	1.5624
4	3.1400	1.4848
5	3.3200	1.6343
6	3.7200	1.7028
7	3.4600	1.6189

1.3.1 Calcul de l'espérance de la Moyenne des échantillons mxs et de l'écart-type stdxS pour chaque échantillon n (à travers les NR réalisations) :

```
[93]: # Estimation de l'espérance de la moyenne des échantillons
E_mxS = np.mean(mxS)

# Écart-type de la moyenne des échantillons
std_mxS = np.std(mxS, ddof=1)

print(f"\nEspérance estimée (E_mxS) : {E_mxS:.4f}")
print(f"Variation de la moyenne (std_mxS) : {std_mxS:.4f}")
```

Espérance estimée (E_mxS) : 3.4698
Variation de la moyenne (std_mxS) : 0.2189

1.4 Conclusion

Ergodicité : Comme mentionné à la page 10, si les valeurs calculées dans l'espace des échantillons sont identiques (ou très proches) à celles calculées dans l'espace des réalisations, cela indique l'ergodicité du premier ordre du processus.

Différence d'axe :

Moments par réalisation : axis=1 (moyenne d'une ligne).

Moments par échantillon : axis=0 (moyenne d'une colonne) .

1.5 Moments du deuxième ordre : la covariance

1.5.1 1. Calcul de la Matrice de Covariance

En Python avec NumPy, pour obtenir le calcul de la covariance entre les colonnes de la matrice résultat sur une matrice où les lignes sont des réalisations, nous utilisons `rowvar=False`

```
[94]: from mpl_toolkits.mplot3d import Axes3D

# Calcul de la matrice de covariance Cx (espace des échantillons)
# rowvar=False signifie que chaque colonne est une variable (échantillon)
Cx = np.cov(x, rowvar=False) # Correspond à  $Cx = cov(x)$ 
```

1.5.2 2. Visualisation 3D (Équivalent de surf)

Pour reproduire le graphique `surf(Cx)` présent à la page 11 de votre document , nous utilisons la fonction `plot_surface` de Matplotlib.

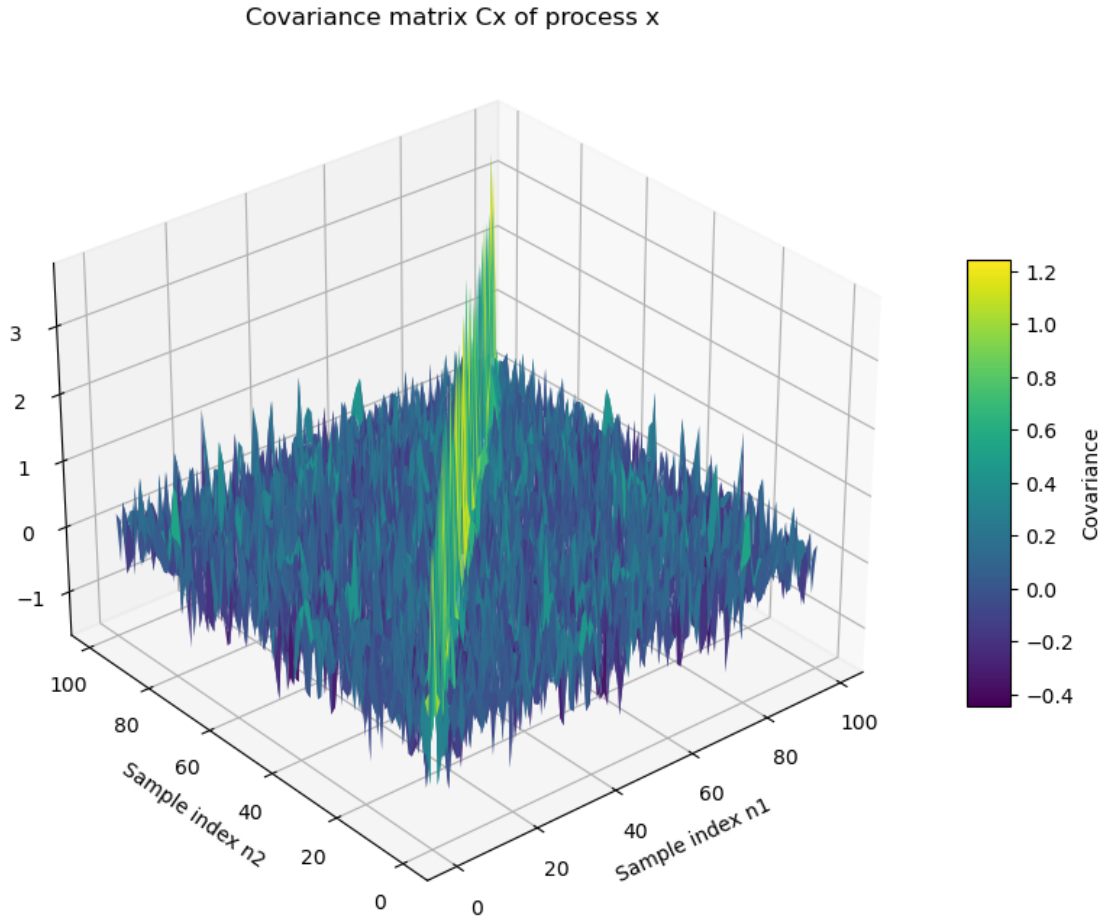
```
[95]: # Création de la grille pour les axes X et Y (indices des échantillons de 0 à N-1)
      N_range = np.arange(N)
      X_mesh, Y_mesh = np.meshgrid(N_range, N_range)

      fig = plt.figure(figsize=(12, 8))
      ax = fig.add_subplot(111, projection='3d')

      # Tracé de la surface (équivalent surf)
      surf = ax.plot_surface(X_mesh, Y_mesh, Cx, cmap='viridis', edgecolor='none')
      # Ajout d'une barre de couleur pour l'échelle de la covariance
      fig.colorbar(surf, shrink=0.5, aspect=10, label='Covariance')
      # Configuration des labels et du titre
      ax.set_title('Covariance matrix Cx of process x')
      ax.set_xlabel('Sample index n1')
      ax.set_ylabel('Sample index n2')
      ax.set_zlabel('Covariance')

      # Ajustement de la vue pour correspondre au document
      ax.view_init(elev=30, azimuth=230)

      plt.show()
```



1.5.3 Observations et Analyse:

Diagonale dominante : On observe une crête très marquée sur la diagonale ($n1=n2$), ce qui correspond à la variance des échantillons .

Incorrélation : En dehors de la diagonale, les valeurs sont proches de zéro (bruit de fond), ce qui indique que les échantillons du processus x (lancer de dés) sont clairement non corrélés entre eux.

Dimensions : La matrice C_x est de taille $N \times N$ (100×100) .

1.6 1. Calcul de la Matrice de Covariance C_{xR}

En Python, comme nous avons défini x avec les réalisations en lignes ($NR \times N$), la fonction `np.cov(x)` calcule par défaut la covariance entre ces lignes donc entre réalisations. Cependant, pour obtenir la covariance entre les échantillons (colonnes), nous devons spécifier `rowvar=False` (comme vu précédemment).

```
[96]: # Calcul de la matrice de covariance CxR (espace des réalisations)
# En Python, np.cov(x) traite chaque ligne comme une variable par défaut
CxR = np.cov(x) # Résultat : matrice NR x NR (50 x 50)
```

Affichage en 3D de la matrice de covariance C_{xR} :

```
[97]: # Création de la grille pour les axes X et Y (indices des réalisations de 0 à
↳ NR-1)
NR_range = np.arange(NR)
X_mesh_R, Y_mesh_R = np.meshgrid(NR_range, NR_range)

fig = plt.figure(figsize=(12, 8))
ax = fig.add_subplot(111, projection='3d')

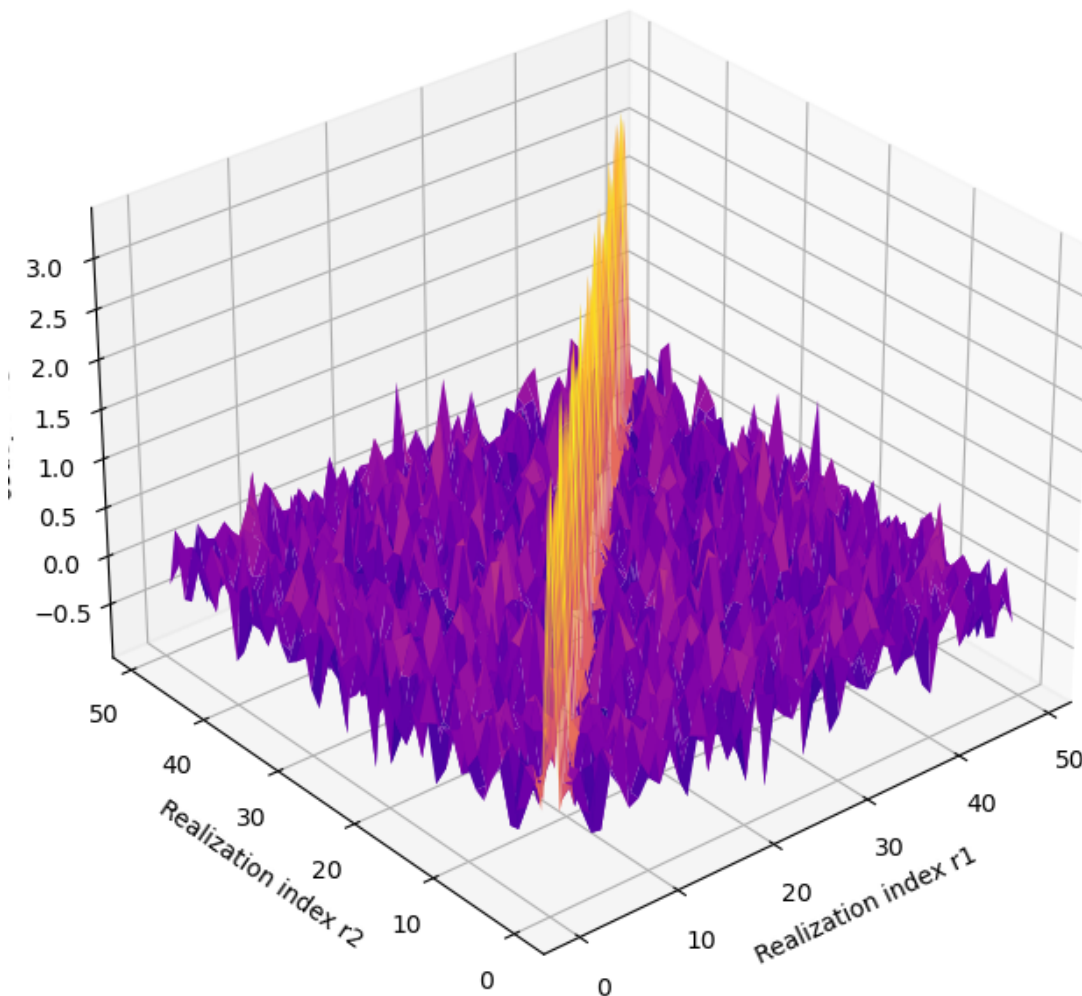
# Tracé de la surface 3D
ax.plot_surface(X_mesh_R, Y_mesh_R, CxR, cmap='plasma', edgecolor='none')

# Configuration des labels et du titre
ax.set_title('covariance matrix Cx of process x along the realizations')
ax.set_xlabel('Realization index r1')
ax.set_ylabel('Realization index r2')
ax.set_zlabel('Covariance')

# Ajustement de la vue pour correspondre aux graphiques du document
ax.view_init(elev=30, azim=230)

plt.show()
```

covariance matrix C_x of process x along the realizations



1.6.1 Analyse des résultats :

Incorrélation des réalisations : Tout comme pour les échantillons temporels, on observe une diagonale dominante . Les valeurs hors-diagonale sont quasi-nulles, confirmant que chaque réalisation du processus est indépendante des autres .

Dimensions : La matrice C_{xR} est de taille $NR \times NR$ (50×50), car nous analysons la corrélation entre les 50 dés . ?

1.7 Take the process y and do the same as before for x ,

Compute the covariance matrix C_y of the process y with the function `covar()`, the argument must be a matrix with a column represent a realization and a line the sample values for each realization for

N samples and NR realizations the matrix must be $N \times NR$. Cy is $NR \times NR$ symmetric (his transpose is equal to himself) , and NR values of the diagonal $Cy(i,i)$ are big compared to the other values why ?

```
[98]: # Calcul de la matrice de covariance Cy (échantillons)
# rowvar=False car les échantillons sont en colonnes dans le calcul statistique
Cy = np.cov(y, rowvar=False) # Correspond à cy = cov(y)

# Visualisation 3D (équivalent à surf)
N_range = np.arange(N)
X, Y = np.meshgrid(N_range, N_range)

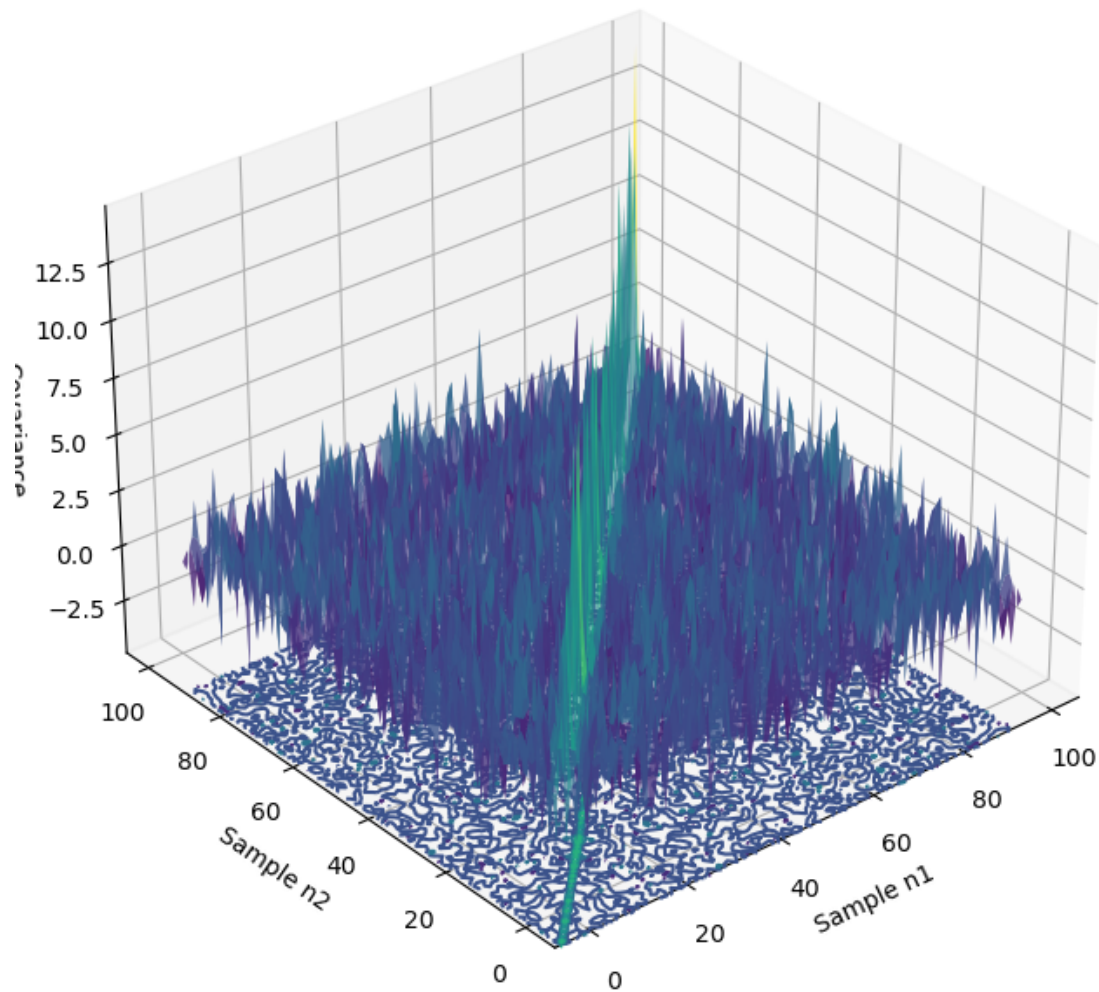
fig = plt.figure(figsize=(12, 8))
ax = fig.add_subplot(111, projection='3d')

# plot_surface avec un contour pour simuler surf()
surf = ax.plot_surface(X, Y, Cy, cmap='viridis', edgecolor='none', alpha=0.8)
cset = ax.contour(X, Y, Cy, zdir='z', offset=np.min(Cy)-2, cmap='viridis')

ax.set_title('covariance matrix Cy of process y')
ax.set_xlabel('Sample n1')
ax.set_ylabel('Sample n2')
ax.set_zlabel('Covariance')
ax.view_init(elev=30, azim=230)

plt.show()
```


covariance matrix C_y of process y



1.8 2. Matrice de covariance C_{yR} et visualisation 3D (Espace des réalisations)

Cette matrice de taille $NR \times NR$ (50×50) montre la corrélation entre les différentes séries générées

```
[99]: # Calcul de la matrice de covariance  $C_{yR}$  (réalisations)
# np.cov(y) calcule la covariance entre les lignes (réalisations)
CyR = np.cov(y)

# Visualisation 3D
NR_range = np.arange(NR)
XR, YR = np.meshgrid(NR_range, NR_range)
```

```

fig = plt.figure(figsize=(12, 8))
ax = fig.add_subplot(111, projection='3d')

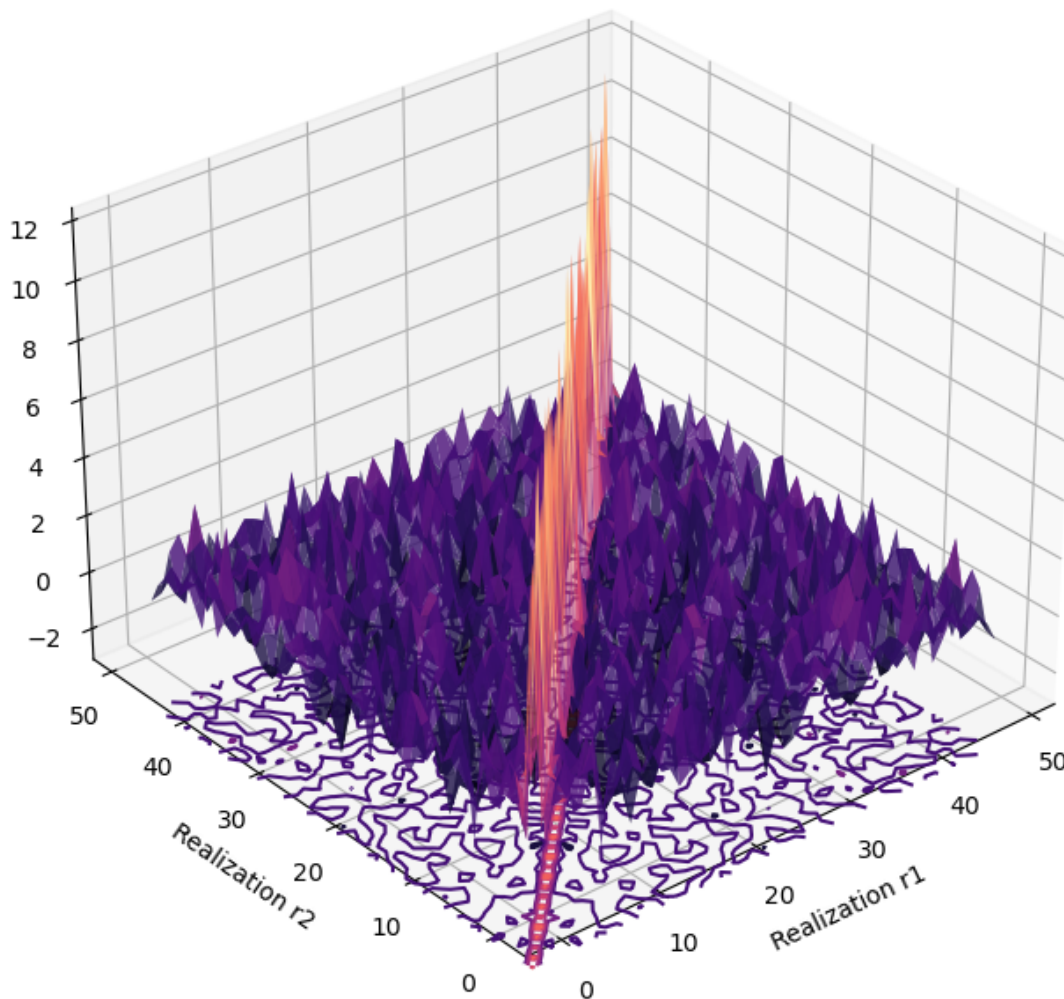
surfR = ax.plot_surface(XR, YR, CyR, cmap='magma', edgecolor='none', alpha=0.8)
csetR = ax.contour(XR, YR, CyR, zdir='z', offset=np.min(CyR)-2, cmap='magma')

ax.set_title('covariance matrix Cy of process y over realizations')
ax.set_xlabel('Realization r1')
ax.set_ylabel('Realization r2')
ax.set_zlabel('Covariance')
ax.view_init(elev=30, azim=230)

plt.show()

```

covariance matrix C_y of process y over realizations



1.9 Analyse des résultats :

Diagonale de C_y : Les valeurs sur la diagonale sont élevées (autour de $A^2 = 9$, ce qui correspond à la puissance du signal (variance) .

Incorrélation temporelle : Comme pour le processus x, les valeurs hors-diagonale de C_y sont très faibles, confirmant que le processus y est un bruit blanc gaussien (échantillons non corrélés entre eux) .

Incorrélation entre réalisations : La matrice $C_{\{yR\}}$ montre également que chaque réalisation du processus est indépendante des autres.

1.10 Filter a random process

- a) If we filter the process y by a linear filter FIR with 10 coefficients for an moving average MA low pass. Or
- b) If we filter the process y by a linear filter IIR with coefficients given by the Butterworth approximation of a second order lowpass filter with $W_n = 0.1$.

1.10.1 1) Let define the FIR filter coefficients for a moving average low pass filter with 10 coefficients:

$b = \text{np.ones}(10) / 10$ # Coefficients du filtre FIR (moyenne mobile sur 10 échantillons) $a = 1$ # Coefficients du filtre IIR (pas de rétroaction pour un filtre FIR)

```
[100]: # --- Définition du filtre FIR MA 10 taps ---  
# Numérateur : [0.1, 0.1, ..., 0.1] (10 fois)  
num_ma = np.ones(10) / 10  
# Dénominateur : [1] pour un filtre FIR  
den_ma = [1]  
  
# Affichage de la réponse impulsionnelle théorique  
print(f"Coefficients du filtre FIR : {num_ma}")
```

Coefficients du filtre FIR : [0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1]

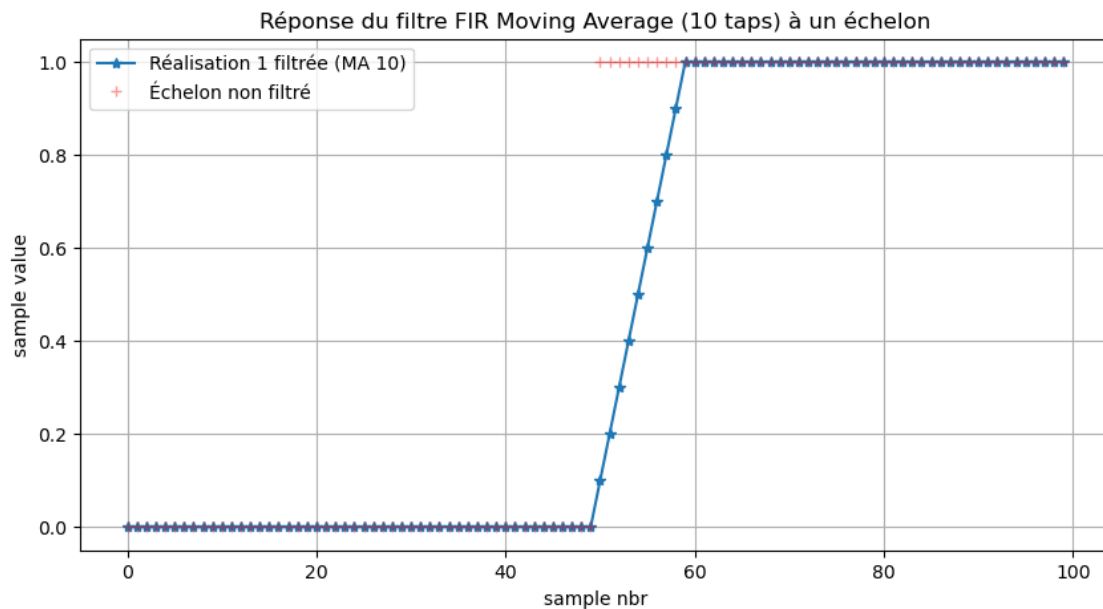
1.10.2 2. Test du filtre avec un échelon unité

Nous modifions la première réalisation du processus y pour y injecter un échelon afin de visualiser la réponse du filtre.

```
[101]: # Préparation du signal de test (réalisation 1 = échelon à n=50)  
y_step_test = y.copy()  
y_step_test[0, :] = np.concatenate([np.zeros(N//2), np.ones(N//2)])  
  
# Application du filtrage FIR  
# yf_ma est la matrice des signaux filtrés  
yf_ma = signal.lfilter(num_ma, den_ma, y_step_test, axis=1)
```

```
# --- Visualisation de la réponse (équivalent page 15 du PDF) ---
plt.figure(figsize=(10, 5))
plt.plot(np.arange(N), yf_ma[0, :], '-*', label='Réalisation 1 filtrée (MA 10)')
plt.plot(np.arange(N), y_step_test[0, :], 'r+', label='Échelon non filtré',
         alpha=0.4)

plt.title('Réponse du filtre FIR Moving Average (10 taps) à un échelon')
plt.xlabel('sample nbr')
plt.ylabel('sample value')
plt.legend()
plt.grid(True)
plt.show()
```



1.10.3 3. Analyse de l'Autocorrélation du signal filtré

Le passage par un filtre MA modifie radicalement l'autocorrélation du bruit blanc (processus y). Pour un filtre MA de 10 points, l'autocorrélation théorique prend une forme triangulaire d'une largeur de 10 échantillons

```
[102]: yf_ma = signal.lfilter(num_ma, den_ma, y, axis=1) # y est le processus original

# Fonction pour simuler le xcorr de MATLAB (avec lags)
def xcorr_python(a, b):
    # On centre les signaux (soustraction de la moyenne) pour la corrélation
    a_centered = a - np.mean(a)
    b_centered = b - np.mean(b)
```

```

corr = signal.correlate(a_centered, b_centered, mode='full')
lags = np.arange(-len(a) + 1, len(a))
return corr, lags

# 1. Corrélation entre 2 instants (échantillons 1 et 2) à travers toutes les
↳ réalisations
# En MATLAB : xcorr(yf(:,1), yf(:,2))
ryf12R, lagR = xcorr_python(yf_ma[:, 0], yf_ma[:, 1])

# 2. Autocorrélation pour une seule réalisation (la n°2)
# En MATLAB : xcorr(yf(2,:), yf(2,:))
ryfyf22, lag = xcorr_python(yf_ma[1, :], yf_ma[1, :])

# 3. Corrélation entre deux réalisations filtrées différentes (la n°4 et la n°7)
# En MATLAB : xcorr(yf(4,:), yf(7,:))
ryfyf47, _ = xcorr_python(yf_ma[3, :], yf_ma[6, :])

# 4. Corrélation entre l'échantillon non filtré et filtré pour la même
↳ réalisation (n°2)
# En MATLAB : xcorr(y(2,:), yf(2,:))
ryyf22, _ = xcorr_python(y[1, :], yf_ma[1, :])

# 5. Autocorrélation du signal non filtré (pour comparaison)
# En MATLAB : xcorr(y(2,:), y(2,:))
ryy22, _ = xcorr_python(y[1, :], y[1, :])

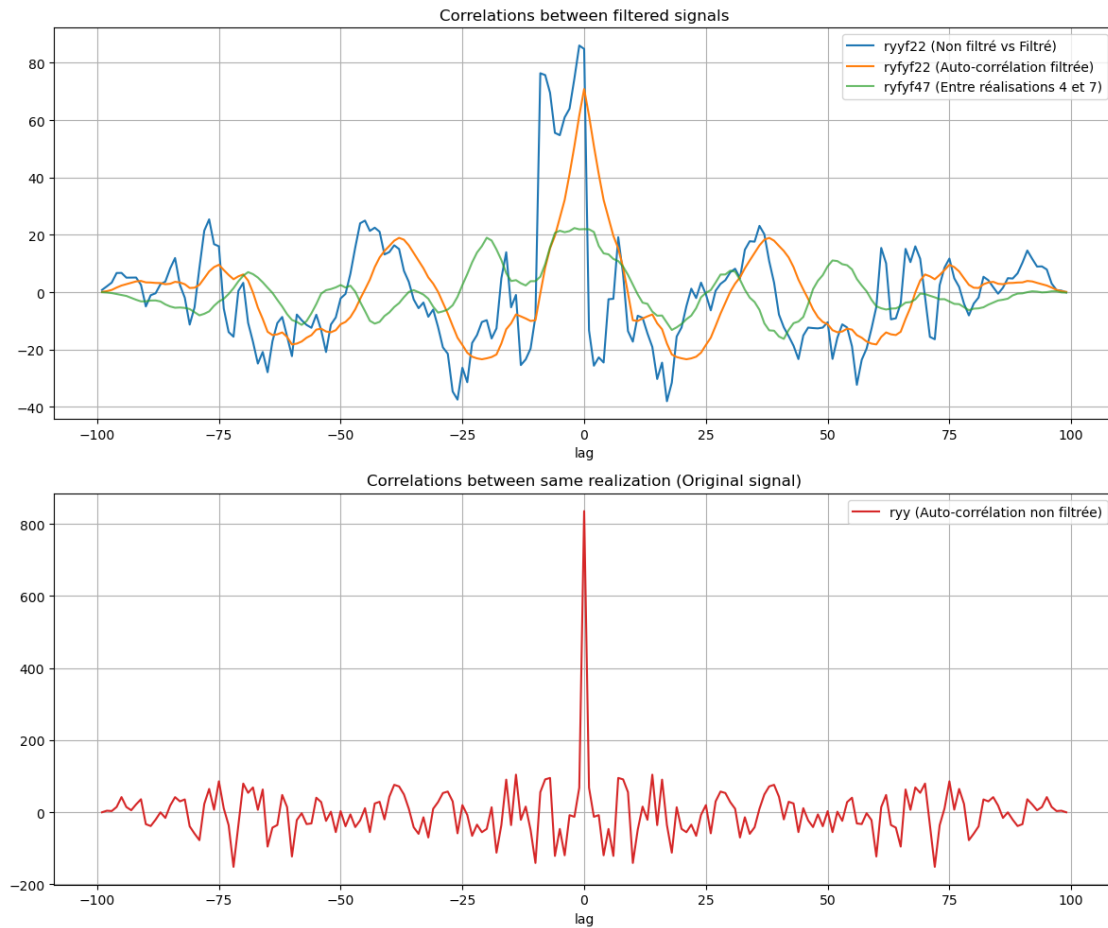
# --- Affichage des graphiques (Comme à la page 16 du PDF) ---
plt.figure(figsize=(12, 10))

# Premier sous-graphique : Corrélations du signal filtré
plt.subplot(2, 1, 1)
plt.plot(lag, ryyf22, label='ryyf22 (Non filtré vs Filtré)')
plt.plot(lag, ryfyf22, label='ryfyf22 (Auto-corrélation filtrée)')
plt.plot(lag, ryfyf47, label='ryfyf47 (Entre réalisations 4 et 7)', alpha=0.7)
plt.title('Correlations between filtered signals')
plt.xlabel('lag')
plt.legend()
plt.grid(True)

# Deuxième sous-graphique : Autocorrélation du signal original
plt.subplot(2, 1, 2)
plt.plot(lag, ryy22, color='tab:red', label='ryy (Auto-corrélation non filtrée)')
plt.title('Correlations between same realization (Original signal)')
plt.xlabel('lag')
plt.legend()
plt.grid(True)

```

```
plt.tight_layout()
plt.show()
```



1.11 Analyse des résultats :

Lissage des corrélations : Vous remarquerez que les courbes du signal filtré (ryfyf22) sont beaucoup plus lisses et larges que celle du signal original (ryy22) . C'est l'effet direct du filtre à moyenne mobile qui introduit une dépendance temporelle entre les échantillons.

Pic d'autocorrélation : Le signal original présente un pic très fin à $\text{lag} = 0$ (caractéristique du bruit blanc), tandis que le signal filtré a un pic plus étalé .

Indépendance des réalisations : La courbe ryfyf47 (entre deux réalisations différentes) reste centrée autour de zéro, confirmant que même après filtrage, les réalisations restent décorrélées entre elles.

1.12 Visualisation de la modification de la matrice de covariance du processus filtré

Cette étape permet de visualiser comment le filtrage introduit une corrélation entre les échantillons temporels tout en préservant l'indépendance des réalisations. ### 1. Calcul des Matrices de

Covariance du Processus Filtré Nous utilisons la matrice `yf_ma` obtenue précédemment (le processus Gaussien y passé à travers le filtre à moyenne mobile de 10 points)

```
[103]: # 1. Matrice de covariance dans l'espace des échantillons (Cyf)
# Correspond à cyf = cov(yf) en MATLAB
# Taille : N x N (100 x 100)
Cyf_ma = np.cov(yf_ma, rowvar=False)

# 2. Matrice de covariance dans l'espace des réalisations (CyfR)
# Correspond à CyfR = cov(yf') en MATLAB
# Taille : NR x NR (50 x 50)
CyfR_ma = np.cov(yf_ma)
```

1.13 2. Visualisation 3D : Espace des Échantillons (C_{yf})

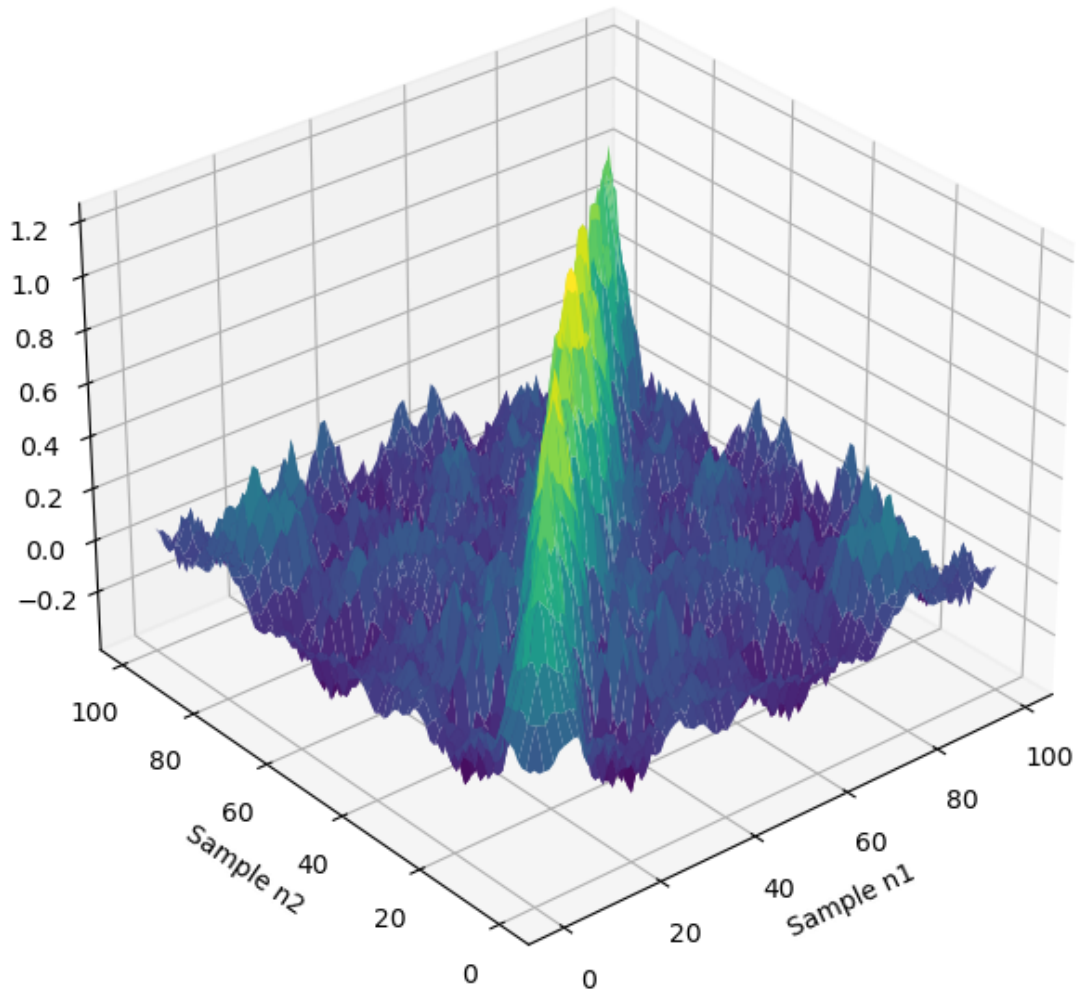
Cette représentation montre l'étalement de la corrélation autour de la diagonale, signe que les échantillons voisins sont désormais corrélés par le filtre.

```
[104]: # Grille pour les échantillons (100x100)
X_f, Y_f = np.meshgrid(np.arange(N), np.arange(N))

fig = plt.figure(figsize=(12, 8))

# --- Graphique 1 : Covariance Espace Échantillons ---
ax1 = fig.add_subplot(1, 2, 1, projection='3d')
surf1 = ax1.plot_surface(X_f, Y_f, Cyf_ma, cmap='viridis', edgecolor='none')
ax1.set_title('Covariance matrix Cyf (FIR MA 10)\nSample Space')
ax1.set_xlabel('Sample n1')
ax1.set_ylabel('Sample n2')
ax1.view_init(elev=30, azim=230)
plt.tight_layout()
plt.show()
```

Covariance matrix C_{yf} (FIR MA 10)
Sample Space



1.14 3. Visualisation 3D : Espace des Réalisations (C_{y_fR})

Cette représentation confirme que les différentes réalisations restent indépendantes les unes des autres malgré le filtrage.

```
[105]: # Grille pour les réalisations (50x50)
XR_f, YR_f = np.meshgrid(np.arange(NR), np.arange(NR))

# --- Graphique 2 : Covariance Espace Réalisations ---
```

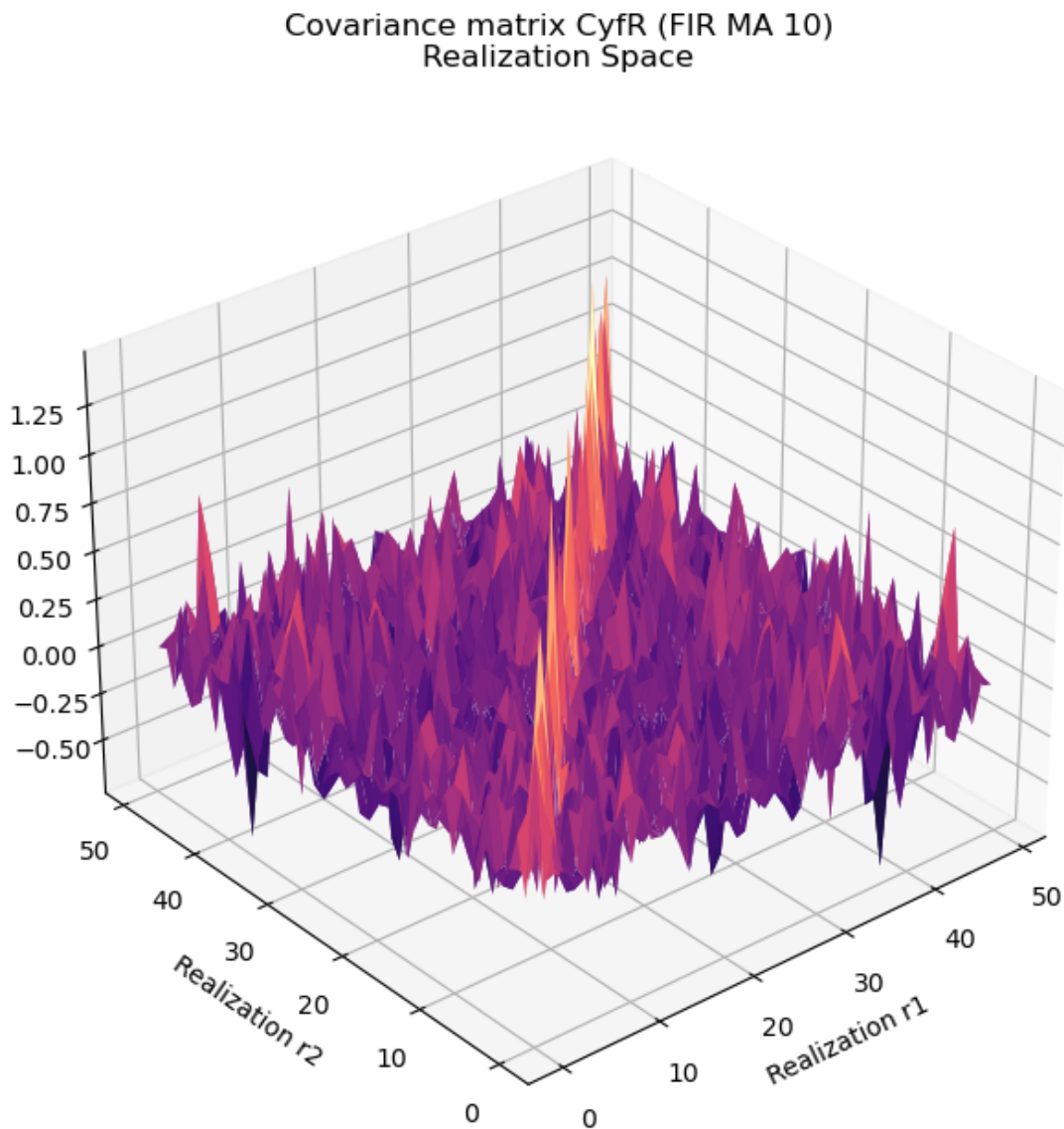


```

fig = plt.figure(figsize=(12, 8))
ax2 = fig.add_subplot(1, 2, 2, projection='3d')
surf2 = ax2.plot_surface(XR_f, YR_f, CyfR_ma, cmap='magma', edgecolor='none')
ax2.set_title('Covariance matrix CyfR (FIR MA 10)\nRealization Space')
ax2.set_xlabel('Realization r1')
ax2.set_ylabel('Realization r2')
ax2.view_init(elev=30, azim=230)

plt.tight_layout()
plt.show()

```



1.15 Analyse Comparative

Corrélation Temporelle (C_{yf}) : Contrairement au bruit blanc original où la covariance était nulle hors diagonale, on observe ici une structure “élargie” le long de la diagonale. Cela prouve que le filtre MA 10 introduit une mémoire dans le signal : chaque échantillon dépend des 9 précédents.

Indépendance (C_{yfR}) : La matrice de covariance des réalisations reste très proche d’une matrice diagonale. Cela signifie que filtrer chaque cas (réalisation) séparément ne crée pas de lien statistique entre les cas.

1.16 Filtrage du processus y avec un filtre IIR Butterworth de second ordre

1.16.1 1. Définition des coefficients du filtre IIR Butterworth

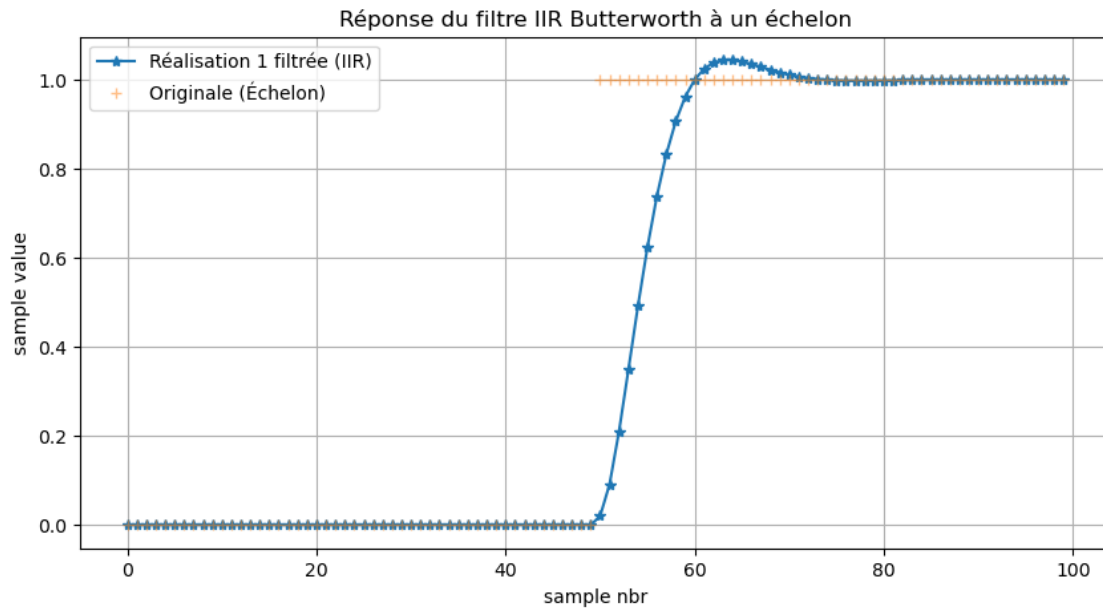
Nous allons utiliser la fonction `butter` de SciPy pour obtenir les coefficients du filtre IIR Butterworth de second ordre avec une fréquence de coupure normalisée $W_n = 0.1$ (correspondant à 0.1 fois la fréquence d’échantillonnage). Tester le filtre avec un échelon unité et analyser la réponse du filtre.

```
[106]: # --- Paramètres du filtre IIR ---
Wn = 0.1
Num, Den = signal.butter(2, Wn, 'low')

# --- Test du filtre avec un échelon (Page 15) ---
y_step_test = y.copy()
y_step_test[0, :] = np.concatenate([np.zeros(N//2), np.ones(N//2)])

# Application du filtre sur la réalisation test
yf_test = signal.lfilter(Num, Den, y_step_test[0, :]) #

plt.figure(figsize=(10, 5))
plt.plot(yf_test, '-*', label='Réalisation 1 filtrée (IIR)')
plt.plot(y_step_test[0, :], '+', label='Originale (Échelon)', alpha=0.5)
plt.title('Réponse du filtre IIR Butterworth à un échelon')
plt.xlabel('sample nbr')
plt.ylabel('sample value')
plt.legend()
plt.grid(True)
plt.show()
```



1.17 Filtrage du processus y avec un filtre IIR Butterworth de second ordre

```
[107]: # Filtrage de la matrice complète (axis=1 pour traiter chaque réalisation)
yf_iir = signal.lfilter(Num, Den, y, axis=1)
```

1.18 Analyse des corrélations du signal filtré avec un filtre IIR Butterworth de second ordre

```
[108]: def xcorr_python(a, b):
    a_centered = a - np.mean(a)
    b_centered = b - np.mean(b)
    corr = signal.correlate(a_centered, b_centered, mode='full')
    lags = np.arange(-len(a) + 1, len(a))
    return corr, lags

# a) Corrélation entre 2 instants (échantillons 1 et 2) sur toutes les
#    réalisations
ryf12R, lagR = xcorr_python(yf_iir[:, 0], yf_iir[:, 1])

# b) Autocorrélation de la réalisation 2 filtrée
ryfyf22, lag = xcorr_python(yf_iir[1, :], yf_iir[1, :])

# c) Corrélation entre deux réalisations filtrées différentes (4 et 7)
ryfyf47, _ = xcorr_python(yf_iir[3, :], yf_iir[6, :])

# d) Corrélation entre le signal non filtré et filtré (réalisation 2)
```

```

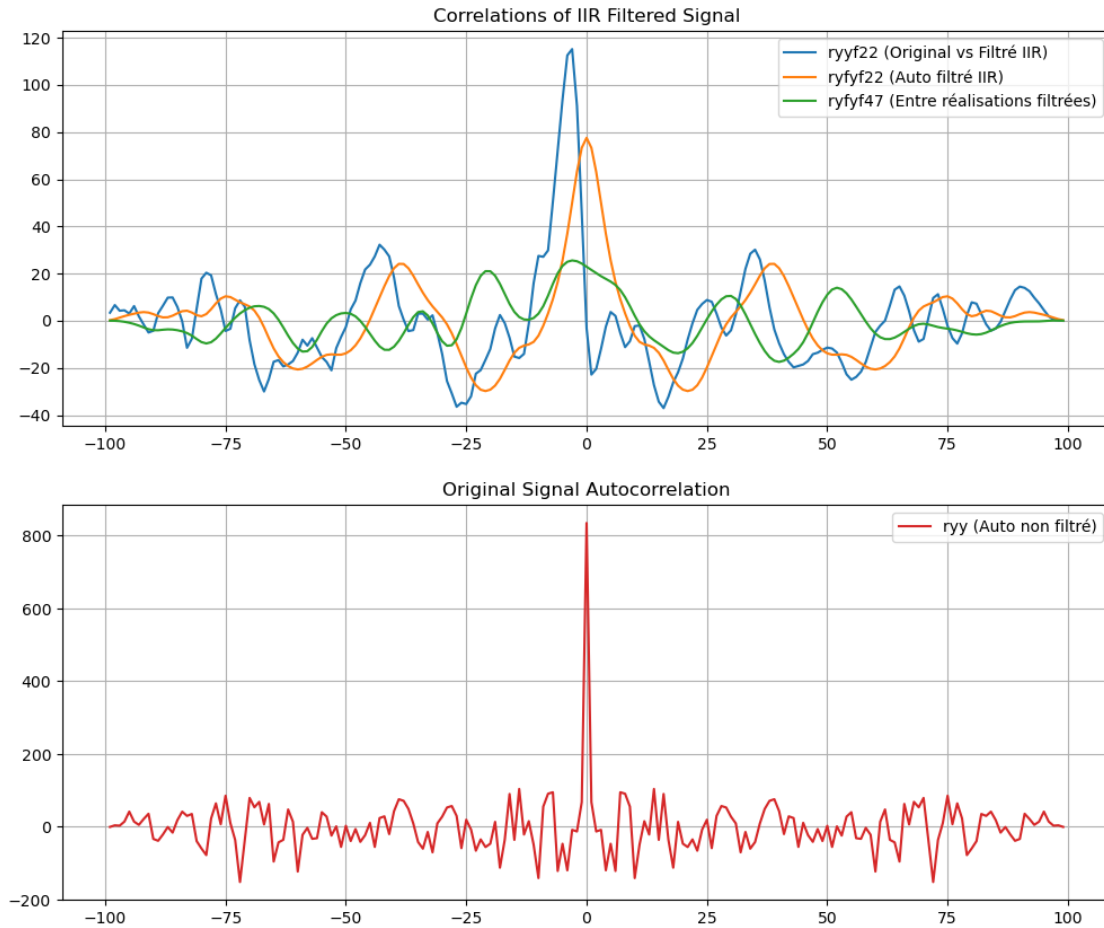
ryyf22, _ = xcorr_python(y[1, :], yf_iir[1, :])

# e) Autocorrélation du signal original (non filtré) pour comparaison
ryy22, _ = xcorr_python(y[1, :], y[1, :])

# --- Affichage (équivalent Figure page 16) ---
plt.figure(figsize=(12, 10))
plt.subplot(2, 1, 1)
plt.plot(lag, ryyf22, label='ryyf22 (Original vs Filtré IIR)')
plt.plot(lag, ryfyf22, label='ryfyf22 (Auto filtré IIR)')
plt.plot(lag, ryfyf47, label='ryfyf47 (Entre réalisations filtrées)')
plt.title('Correlations of IIR Filtered Signal')
plt.legend()
plt.grid(True)

plt.subplot(2, 1, 2)
plt.plot(lag, ryy22, color='tab:red', label='ryy (Auto non filtré)')
plt.title('Original Signal Autocorrelation')
plt.legend()
plt.grid(True)
plt.show()

```



1.19 Matrices de Covariance 3D

Enfin, nous visualisons les matrices de covariance pour voir l'impact de la réponse du filtre IIR sur la structure statistique du processus

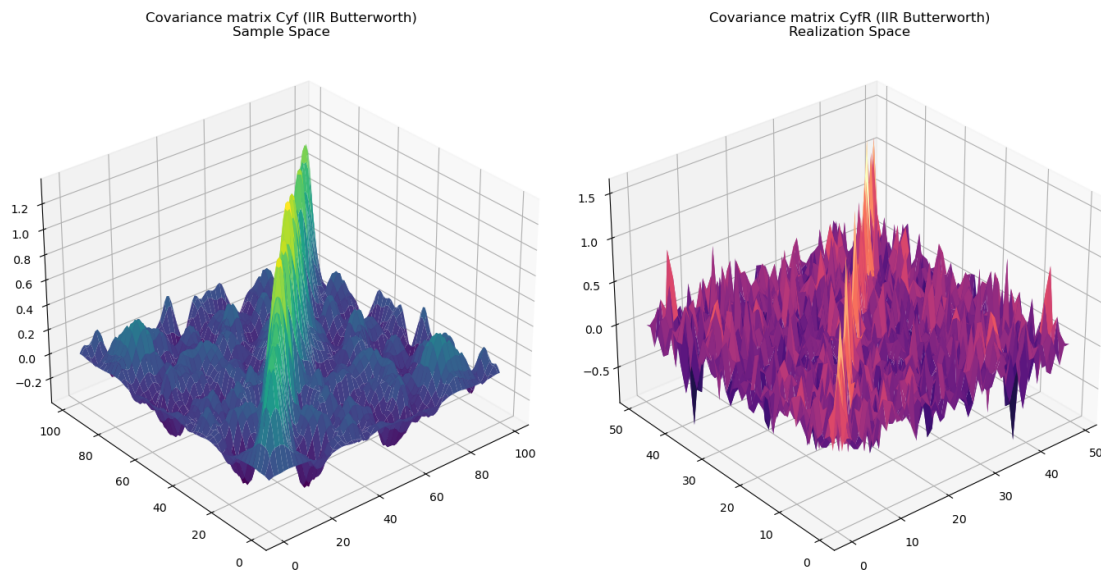
```
[109]: # Calcul des matrices
Cyf_iir = np.cov(yf_iir, rowvar=False) # Espace des échantillons
CyfR_iir = np.cov(yf_iir)              # Espace des réalisations

# --- Affichage 3D ---
fig = plt.figure(figsize=(14, 7))

# Matrice de covariance - Espace Échantillons
ax1 = fig.add_subplot(1, 2, 1, projection='3d')
X, Y = np.meshgrid(np.arange(N), np.arange(N))
ax1.plot_surface(X, Y, Cyf_iir, cmap='viridis', edgecolor='none')
ax1.set_title('Covariance matrix Cyf (IIR Butterworth)\nSample Space')
ax1.view_init(elev=30, azimuth=230)
```

```
# Matrice de covariance - Espace Réalisations
ax2 = fig.add_subplot(1, 2, 2, projection='3d')
XR, YR = np.meshgrid(np.arange(NR), np.arange(NR))
ax2.plot_surface(XR, YR, CyfR_iir, cmap='magma', edgecolor='none')
ax2.set_title('Covariance matrix CyfR (IIR Butterworth)\nRealization Space')
ax2.view_init(elev=30, azim=230)

plt.tight_layout()
plt.show()
```



1.20 Observations spécifiques au filtre IIR:

Réponse temporelle : Contrairement au filtre FIR qui est monotone, l'échelon montre ici un dépassement (overshoot) et des oscillations avant stabilisation.

Structure de Corrélation : Dans la matrice C_{yf} : (espace échantillons), la diagonale est plus "riche" en relief. On observe des zones de corrélation négative légère autour de la diagonale, reflets des oscillations de la réponse impulsionnelle du filtre de Butterworth .

Indépendance : La matrice C_{yfR} reste diagonale, confirmant que les réalisations restent statistiquement indépendantes.